uerying

#2 SQL Development	#10 Data-Driven Decision Making	DML / DDL	JOINs	Window Functions

● Day Focus

Day 2 is the most visible skills day — participants will use these queries in their actual NSR work and share them with colleagues. Progress through the teaching sequence in order: DML/DDL first, JOINs second, window functions third. Never jump ahead. Each section builds on vocabulary from the previous one, and the exercises at the end weave all three together.

Contents

Pre-Session Checklist

- The nsr schema must already exist from Day 1 — confirm with: \dn in psql
- Create the nsr.payments table (DDL below) before participants arrive — it is needed for Section 2 and 3 demos
- Insert a small set of payment rows for the demo (sample INSERT below)
- Open the platform at Day 2 and confirm all three tabs load: DML/DDL, JOINs, Window Functions
- Log in as Instructor (PIN: AUKTIE2026) — confirm the Day 2 Mark Complete button is visible
- Prepare a single psql terminal window on the projector — Day 2 does not use two-terminal demos
- Remind participants at the start: Day 2 builds on the schema created in Day 1 — they need the same PostgreSQL connection

Pre-session: Create the payments table

```sql
-- Run this before the session — required for Section 2 and 3 demos
CREATE TABLE IF NOT EXISTS nsr.payments (
payment_id BIGSERIAL PRIMARY KEY,
household_id BIGINT NOT NULL REFERENCES nsr.households(household_id),
payment_date DATE NOT NULL,
amount_ngn NUMERIC(12,2) NOT NULL CHECK (amount_ngn > 0),
status TEXT NOT NULL CHECK (status IN ('pending','processed','failed'))
);

-- Insert sample payments for the demo
INSERT INTO nsr.payments (household_id, payment_date, amount_ngn, status)
VALUES
(1, '2026-01-15', 5000.00, 'processed'),
(1, '2026-02-15', 5000.00, 'processed'),
(1, '2026-03-15', 5000.00, 'failed'),
(2, '2026-01-20', 5000.00, 'processed');

-- Confirm
SELECT * FROM nsr.payments;
```

09:30 — Registration & Day 1 Recap

■ 09:30 Doors open ■ 10:00 Training starts

Open with a 5-minute verbal recap of Day 1. Ask the room: *'What is MVCC? What does WAL protect against?'* — cold-call two participants. This is a low-stakes temperature check, not an examination. If the room is uncertain, spend an extra 5 minutes on a summary before moving to Day 2 content.

Day 2 assumption: every participant has the nsr schema with states, lgas, households, and individuals populated from Day 1. If anyone is missing data, direct them to re-run the Day 1 setup SQL silently while you

continue with the rest of the room.

⟹ Section 1 — DDL & DML

~60 mins

● What DDL and DML mean — establish this before the first demo

DDL (Data Definition Language) is the set of SQL commands that define or change the structure of the database: CREATE TABLE, ALTER TABLE, DROP TABLE, TRUNCATE. In PostgreSQL, DDL commands are transactional — you can roll them back within a BEGIN/COMMIT block. This is unusual: in MySQL or Oracle, DDL causes an implicit commit and cannot be undone.

DML (Data Manipulation Language) is the set of commands that read or modify data: SELECT, INSERT, UPDATE, DELETE. DML operations are always subject to MVCC visibility rules and can always be wrapped in transactions.

1.1 — INSERT with RETURNING

Demonstrate INSERT RETURNING. Ask the room: *'Why would you need the generated household_id immediately after inserting?'* Answer: to insert the household members (individuals) in the same script — you need the ID before you can reference it.

```sql
-- INSERT with RETURNING — gets the generated BIGSERIAL id immediately
INSERT INTO nsr.households (household_code, lga_id, head_name, pmt_score)
VALUES ('HH-TEST-001', 1, 'Amina Musa', 22.750)
RETURNING household_id, household_code;
```

● RETURNING — unique to PostgreSQL

RETURNING is not part of standard SQL and is not available in MySQL. It attaches a SELECT clause to an INSERT or UPDATE, returning columns from the rows that were just written — without requiring a second round-trip query to retrieve them.

In NSR workflows this is essential: when you insert a new household, PostgreSQL generates the household_id (a BIGSERIAL). RETURNING gives you that ID instantly in the same statement. Your script can immediately use it to insert that household's individuals — no second query, no race condition between two separate statements.

1.2 — UPDATE with RETURNING

```sql
-- UPDATE with RETURNING — confirms exactly which rows changed and what the new values
are
UPDATE nsr.households
SET pmt_score = 30.250, enumeration_round = 2
WHERE household_code = 'HH-TEST-001'
RETURNING household_id, household_code, pmt_score;
```

● Why RETURNING on UPDATE matters

Without RETURNING, an UPDATE returns only a row count: 'UPDATE 3'. You do not know which rows were affected or what their new values are without running a separate SELECT. RETURNING gives you the post-update state of every changed row in a single operation — useful for audit logging, confirming that a score was set correctly, or chaining the updated values into the next step of a script.

1.3 — DELETE safety: SELECT before you DELETE

Before running the DELETE, run the exact same WHERE clause as a SELECT. Show the room what would be deleted before deleting it. Ask: *'What happens if the WHERE clause is wrong and this cascades?'*

```sql
-- Step 1: verify the target — use the exact same WHERE clause you will use for DELETE
SELECT household_id, household_code, head_name
FROM nsr.households
WHERE household_code = 'HH-TEST-001';

-- Step 2: if the result looks correct, proceed with the DELETE
DELETE FROM nsr.households
WHERE household_code = 'HH-TEST-001';
```

● Why SELECT-before-DELETE is mandatory practice in NSR

The households table has ON DELETE CASCADE to individuals. Deleting one household row also deletes every individual record under it. A wrong WHERE clause — for example, accidentally omitting the household_code filter — could cascade-delete thousands of records in a single statement. There is no undo outside a transaction.

The SELECT-first pattern costs one extra second and has prevented countless production disasters. Make it a habit your participants carry into their actual work.

1.4 — TRUNCATE vs DELETE

```sql
-- DELETE removes rows one at a time, fires triggers, can be rolled back
DELETE FROM nsr.households WHERE is_active = FALSE;

-- TRUNCATE removes ALL rows instantly by deallocating storage pages
-- RESTART IDENTITY resets BIGSERIAL sequences back to 1
-- CASCADE extends the truncate to child tables (households -> individuals)
-- WARNING: do not use on production data. Appropriate for staging tables only.
TRUNCATE nsr.households RESTART IDENTITY CASCADE;
```

	DELETE	TRUNCATE
Speed	Slower — deletes row by row, updates indexes	Instant — deallocates whole storage pages
Triggers	Fires row-level DELETE triggers	Does not fire row-level triggers
Rollback	Can be rolled back within a transaction	Can be rolled back only if inside BEGIN/COMMIT

WHERE clause	Supports filtering — delete specific rows	No WHERE — removes all rows always
Sequences	Does not reset BIGSERIAL counters	RESTART IDENTITY resets sequences to 1
Correct use	Targeted row removal in production	Clearing staging/import tables between runs

1.5 — Transaction control: BEGIN / COMMIT / ROLLBACK

Wrap a multi-step DML operation in a transaction to demonstrate atomicity. Then ROLLBACK to show the room that the database reverts completely.

```sql
BEGIN;

-- Update a pmt_score
UPDATE nsr.households SET pmt_score = 99.9 WHERE household_code = 'HH-LA-001';

-- Verify the change is visible inside this transaction
SELECT household_code, pmt_score FROM nsr.households WHERE household_code = 'HH-LA-001';

-- Oops — wrong score. Roll back the whole transaction.
ROLLBACK;

-- The table is unchanged
SELECT household_code, pmt_score FROM nsr.households WHERE household_code = 'HH-LA-001';
```

● Atomicity — all or nothing

A transaction is the database's guarantee of atomicity: either every statement inside the BEGIN/COMMIT block succeeds and is committed together, or the whole thing is rolled back as if none of it happened. This is the 'A' in ACID.

In NSR batch processing this is critical. Imagine a script that (1) inserts a new household, (2) inserts the household members, and (3) marks the source record as processed. If step 2 fails halfway through, a ROLLBACK removes the partial household too — the database stays consistent. Without a transaction, you would have a household with no members and no way to know the source record was already processed.

■ Section 2 — JOINs

~50 mins

● The core JOIN mental model — establish this before any code

A JOIN combines rows from two tables based on a matching column. Think of it as a lookup: 'for each row in table A, find the matching row in table B.' The key question is always: what happens when there is no match?

INNER JOIN: rows with no match are silently dropped from both sides. Use when a match is mandatory — every household must have an LGA.

LEFT JOIN: all rows from the left table are kept. Rows with no match on the right get NULLs in the right-hand columns. Use when absence of a match is meaningful — you want to see households that have never been paid.

2.1 — INNER JOIN: household report across three tables

Run this query and ask: *'Why do we need two JOINs instead of one?'* Draw on the whiteboard: households → lgas → states. State name is not on households (3NF) — it requires two hops.

```sql
-- INNER JOIN: households with their state and LGA
-- Two JOINs needed: households -> lgas gives lga_name; lgas -> states gives state_name
SELECT
h.household_code,
h.head_name,
s.state_name,
l.lga_name,
h.pmt_score
FROM nsr.households h
JOIN nsr.lgas l ON h.lga_id = l.lga_id
JOIN nsr.states s ON l.state_code = s.state_code
WHERE h.is_active = TRUE
ORDER BY s.state_name, h.pmt_score;
```

● Why INNER JOIN is correct here

Every household in the NSR must belong to a valid LGA in a valid state — the foreign key constraint enforces this. If a household had no matching LGA (which the FK prevents), it would represent a data corruption problem. Using INNER JOIN means any such corrupt row would disappear from the result silently, which is actually a useful diagnostic: if the row count from this JOIN is lower than SELECT COUNT(*) FROM nsr.households, something is wrong with the data. In a clean database they will always match.

2.2 — LEFT JOIN: payment summary including unpaid households

Ask before running: *'What value will payment_count show for a household that has never been paid?'* Answer: 0 — because COUNT() on a NULL returns 0. Then ask: *'What will total_paid show?'* Answer: NULL.

```sql
-- LEFT JOIN: all active households with payment totals — zero for those never paid
SELECT
h.household_code,
h.head_name,
COUNT(p.payment_id) AS payment_count,
SUM(p.amount_ngn) AS total_paid
FROM nsr.households h
LEFT JOIN nsr.payments p
ON h.household_id = p.household_id
AND p.status = 'processed'
WHERE h.is_active = TRUE
GROUP BY h.household_id, h.household_code, h.head_name
```

```
  ORDER BY payment_count DESC;
```

● The AND in the ON clause — a critical distinction

The condition **p.status = 'processed'** is inside the ON clause, not the WHERE clause. This is deliberate and important.

Placing it in ON means: 'try to match only to processed payments.' A household with only pending or failed payments finds no processed match, so it appears in the result with payment_count = 0. The LEFT JOIN preserves it.

If you moved p.status = 'processed' to the WHERE clause, PostgreSQL would first do the LEFT JOIN (keeping all households), then filter out any row where status is not 'processed' — which includes the NULL rows for unpaid households. The result would silently drop all households with zero processed payments. This is one of the most common LEFT JOIN bugs in real-world SQL, and it produces no error — just wrong numbers.

2.3 — Anti-join: households that have never received a processed payment

This is the policy query: who has been enrolled but never paid? Ask the room to write it first, then show the solution. Most participants will reach for a subquery — show the LEFT JOIN approach is faster.

sql

```sql
-- Anti-join pattern: LEFT JOIN + IS NULL finds 'households with no match'
SELECT
h.household_code,
h.head_name,
s.state_name
FROM nsr.households h
JOIN nsr.lgas l ON h.lga_id = l.lga_id
JOIN nsr.states s ON l.state_code = s.state_code
LEFT JOIN nsr.payments p
ON p.household_id = h.household_id
AND p.status = 'processed'
WHERE h.is_active = TRUE
AND p.payment_id IS NULL
ORDER BY s.state_name, h.household_code;
```

● The anti-join pattern explained

The LEFT JOIN attempts to match each household to a processed payment. For households that have never been paid, no match is found — the p.* columns come back as NULL. The WHERE condition **p.payment_id IS NULL** keeps only those unmatched rows.

This is called an anti-join: it returns rows from the left table that have no counterpart in the right table. It is semantically equivalent to NOT IN (subquery) or NOT EXISTS, but typically faster because the planner can use a hash join rather than evaluating a correlated subquery for every row.

JOIN type	Rows returned	Use when
INNER JOIN	Only rows with a match on both sides	Match is mandatory — every household must have an LGA

LEFT JOIN	All left rows; NULLs on right where no match	Absence of match is meaningful — who has no payments?
RIGHT JOIN	All right rows; NULLs on left where no match	Rarely used — rewrite as LEFT JOIN with tables swapped
FULL OUTER JOIN	All rows from both sides; NULLs where no match	Comparing two datasets for discrepancies
CROSS JOIN	Every combination of left and right rows	Generating test data; almost never in production

Section 3 — Window Functions

~40 mins

● GROUP BY vs window functions — the key distinction

GROUP BY collapses rows — you get one output row per group and lose all the detail rows. You cannot see individual households once you have grouped by state.

Window functions keep every row — they attach a computed value alongside each row without collapsing anything. You can see every individual household and its rank within its state in the same result set.

The syntax that makes this work is the OVER () clause. OVER () tells PostgreSQL: 'compute this value using a window of rows, not a collapsed group.' PARTITION BY divides rows into independent groups (like GROUP BY but without collapsing). ORDER BY controls the ranking or accumulation order within each partition.

3.1 — RANK() and PERCENT_RANK(): poverty targeting

This is the query participants will most want to take back to work. Run it, then explain the OVER clause column by column. Ask: *'What does rank 1 mean here?'* — the most vulnerable household in that state.

```sql
-- Rank households by pmt_score within their state
-- Lower pmt_score = more vulnerable = rank 1
SELECT
h.household_code,
h.head_name,
s.state_name,
h.pmt_score,
RANK() OVER (
PARTITION BY l.state_code
ORDER BY h.pmt_score ASC
) AS poverty_rank,
ROUND(
100.0 * PERCENT_RANK() OVER (
PARTITION BY l.state_code ORDER BY h.pmt_score
), 1
```

```
  ) AS percentile
  FROM nsr.households h
  JOIN nsr.lgas l ON h.lga_id = l.lga_id
  JOIN nsr.states s ON l.state_code = s.state_code
  WHERE h.is_active = TRUE
  ORDER BY s.state_name, poverty_rank;
```

● Breaking down the OVER clause

PARTITION BY l.state_code: divides all households into separate groups by state. RANK() resets to 1 at the start of each state — Lagos rank 1 and Kano rank 1 are independent.

ORDER BY h.pmt_score ASC: within each state, households are ordered from lowest score to highest. Rank 1 is the most economically vulnerable household in that state.

RANK() vs DENSE_RANK(): if two households tie at rank 3, RANK() produces 3, 3, 5 (skips 4 — the gap signals a tie). DENSE_RANK() produces 3, 3, 4 (no gap). For poverty targeting, RANK() is usually preferred — tied households are treated equally, and the gap signals to analysts that a tie occurred.

PERCENT_RANK(): returns a value from 0.0 to 1.0 representing where the row falls relative to all rows in its partition. Multiplied by 100 it gives a percentile. A household at the 10th percentile is among the most vulnerable 10% in its state.

3.2 — SUM() OVER: cumulative disbursement per household

This shows a running total — a common analytics requirement. Ask: *'Could you do this with GROUP BY?'* No — GROUP BY gives you the total per household, but not the running subtotal after each payment date.

sql

```
  -- Running total of processed payments per household, ordered by date
  SELECT
  household_id,
  payment_date,
  amount_ngn,
  SUM(amount_ngn) OVER (
  PARTITION BY household_id
  ORDER BY payment_date
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
  ) AS cumulative_paid
  FROM nsr.payments
  WHERE status = 'processed'
  ORDER BY household_id, payment_date;
```

● The frame clause: ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW

The frame clause defines exactly which rows within the window are included in each computation. UNBOUNDED PRECEDING means 'from the very first row in this partition.' CURRENT ROW means 'up to and including the row we are on right now.' Together they build a running sum: the first payment row shows just that payment; the second shows the sum of the first and second; and so on.

Without an explicit frame clause, PostgreSQL uses a default that can produce unexpected results when ORDER BY is present. Always write the frame clause explicitly for running totals to make the intent clear.

Function	What it computes	NSR use case
RANK()	Position within partition; ties share a rank, next rank skips	Poverty targeting — rank households by vulnerability within state
DENSE_RANK()	Same as RANK() but no gaps after ties	When analysts need consecutive ranks with no missing numbers
PERCENT_RANK()	Relative position as 0.0–1.0 (percentile)	Identify bottom 10% of households for priority intervention
ROW_NUMBER()	Unique sequential number — no ties, no gaps	Pagination, deduplication, picking one row per group
SUM() OVER	Running or partitioned total without collapsing rows	Cumulative disbursement per household over time
LAG() / LEAD()	Value from the previous / next row in the window	Month-on-month change in pmt_score; next payment date

■ Section 4 — Subqueries & CTEs

~20 mins

● When to use a CTE instead of a subquery

A CTE (Common Table Expression) is a named subquery defined with the WITH keyword at the top of a query. It does not change what the query does — it changes how readable it is. Use a CTE when a subquery is long enough that embedding it inline makes the main query hard to follow, or when you need to reference the same intermediate result more than once.

4.1 — CTE: identify the bottom 20% of households by pmt_score

sql

```sql
-- CTE: name an intermediate result and reference it in the main query
WITH ranked AS (
SELECT
h.household_id,
h.household_code,
h.head_name,
s.state_name,
h.pmt_score,
PERCENT_RANK() OVER (
PARTITION BY l.state_code ORDER BY h.pmt_score
```

```
) AS pct_rank
FROM nsr.households h
JOIN nsr.lgas l ON h.lga_id = l.lga_id
JOIN nsr.states s ON l.state_code = s.state_code
WHERE h.is_active = TRUE
)
SELECT household_code, head_name, state_name, pmt_score, pct_rank
FROM ranked
WHERE pct_rank <= 0.20
ORDER BY state_name, pmt_score;
```

● Why the WHERE on pct_rank must be outside the CTE

Window functions are computed after the WHERE clause in a single query — you cannot filter on a window function result in the same SELECT where it is computed. The CTE solves this by making the ranked result a named intermediate table, which the outer query can then filter freely. This is a very common pattern whenever you need to filter on a RANK(), ROW_NUMBER(), or PERCENT_RANK() value.

4.2 — Correlated subquery vs EXISTS

Show both patterns. Explain that EXISTS stops as soon as it finds the first match — it does not scan the whole subquery result. For large tables this is significantly faster.

sql

```
-- Correlated subquery: households where at least one payment exists
-- Runs the inner SELECT once per outer row — can be slow on large tables
SELECT household_code, head_name
FROM nsr.households h
WHERE (SELECT COUNT(*) FROM nsr.payments p
WHERE p.household_id = h.household_id) > 0;

-- EXISTS is faster: stops at the first match, does not count all rows
SELECT household_code, head_name
FROM nsr.households h
WHERE EXISTS (
SELECT 1 FROM nsr.payments p
WHERE p.household_id = h.household_id
AND p.status = 'processed'
);
```

Exercises & Knowledge Check

Direct participants to the Exercises tab on Day 2 in the platform. Three exercises are available — all use the nsr schema they built yesterday:

Exercise	Level	What to look for
B1 — JOIN-based household report (household_code, head_name, state_name, lga_name, pmt_score)	Basic	Do they use two JOINs? Correct aliases? ORDER BY both columns?
B2 — Window function ranking by pmt_score within state	Intermediate	PARTITION BY and ORDER BY both present? ASC for lowest-first ranking?
B3 — Households never paid using LEFT JOIN (not subquery)	Intermediate	Filter on status in ON clause (not WHERE)? IS NULL check on payment_id?

● Common mistakes to watch for while walking the room

B1: Using only one JOIN and trying to get state_name directly from households. Remind them: 3NF means state_name is not on households — it requires two hops.

B2: Forgetting PARTITION BY and getting a single global rank across all states instead of a per-state rank. Ask them to check whether Lagos rank 1 and Kano rank 1 both appear.

B3: Putting p.status = 'processed' in the WHERE clause instead of the ON clause. This silently converts the LEFT JOIN to an INNER JOIN and drops unpaid households. Ask them to verify: does a household with only failed payments appear in their results?

After exercises, open the Knowledge Check tab. Questions cover JOIN type selection, RETURNING behaviour, TRUNCATE vs DELETE, and window function syntax. Widespread wrong answers on any question signals a concept to revisit at the start of Day 3.

Day 2 Closing Message

● Suggested closing

"The queries you wrote today are the ones you will actually use. The JOIN report, the never-paid list, the vulnerability ranking — these are real NSR analytics. Tomorrow we add the security layer: who is allowed to run these queries, on which tables, and how we audit what they have done. The SQL stays the same; the question is who controls it."

Mark Day 2 complete using the Mark Complete button on the platform. Remind participants that Day 3 covers PostgreSQL Administration, Security, and Backup — they should bring their Day 1 and Day 2 query notes with them.

Common Participant Questions

Q: What is the difference between WHERE and HAVING?

A: WHERE filters rows before grouping — it cannot reference aggregates. HAVING filters groups after grouping — it can reference COUNT(), SUM() etc. If you are not using GROUP BY, you almost always want WHERE.

Q: Can I use RETURNING with DELETE?

A: Yes. DELETE FROM nsr.households WHERE ... RETURNING household_id returns the IDs of the rows that were just deleted. Useful for audit logging or passing deleted IDs to a subsequent INSERT into an archive table.

Q: Why does my LEFT JOIN give fewer rows than expected?

A: Most likely the status filter is in the WHERE clause instead of the ON clause. Moving it to WHERE converts the LEFT JOIN into an INNER JOIN for filtered rows. Check your ON clause carefully.

Q: What is the difference between RANK() and ROW_NUMBER()?

A: ROW_NUMBER() always assigns a unique sequential number — ties get different numbers (arbitrary ordering between tied rows). RANK() gives tied rows the same number and skips subsequent numbers. Use ROW_NUMBER() for deduplication; use RANK() for fair competitive ordering.

Q: Can I wrap a DDL statement in a transaction?

A: Yes — in PostgreSQL, DDL is transactional. You can BEGIN; CREATE TABLE ...; ROLLBACK; and the table will not exist. This is unlike MySQL and Oracle where DDL causes an implicit commit. It makes schema migrations much safer: wrap them in transactions and roll back if anything fails.

Q: What is the performance difference between NOT IN and NOT EXISTS?

A: NOT IN has a dangerous edge case: if any value in the subquery list is NULL, the entire NOT IN returns no rows (NULL comparison logic). NOT EXISTS does not have this problem. On large tables NOT EXISTS is also typically faster because it stops at the first match. Prefer NOT EXISTS or the LEFT JOIN IS NULL anti-join pattern.

Q: When should I use a CTE vs a subquery?

A: Use a CTE when the subquery is long enough to hurt readability inline, when you need to reference the same result more than once, or when you need to filter on a window function result (which requires an outer query). For simple one-use subqueries, an inline subquery is fine.